

Guide de fonctionnement et choix d'implémentation — CDC 751

Ce programme reconstruit les données de caisse de CAMMY FRANCE DEVELOPPEMENT LTD à partir des journaux électroniques (EJ) et des rapports Z, les centralise dans SQLite, puis génère les tableaux de contrôle au format LibreOffice Calc (`.ods`).

Le point d'entrée `traitement` exécute toute la chaîne dans un ordre déterminé. Une exécution complète produit :

- une base SQLite reconstruite et validée ;
- 16 CSV intermédiaires ;
- 30 classeurs ODS de données, de contrôles et de rapprochements ;
- un rapport texte qui trace les sources, les volumes et les totaux de chaque feuille produite.

Le programme ne génère plus de classeurs `.xlsx` ni de rapport PDF.

En bref

Lancer le traitement complet

Pour exécuter le traitement complet dans un conteneur Docker, depuis la racine du projet :

```
docker compose up --build; docker compose down --rmi local
```

Récupérer les résultats

Les résultats sont disponibles **dans le dossier local `output/`** après l'arrêt du service.

Si un dossier `output/` existe déjà, son contenu est copié dans `output/_sauvegarde_<horodatage>/` avant la régénération des fichiers actifs.

Les sources placées dans `fichiers_sources/` ne sont jamais modifiées.

Docker est le choix de traitement par défaut. Il fournit un environnement Python 3.12 avec LibreOffice et PyUNO, sans dépendre de l'installation locale de l'utilisateur. Il suffit d'avoir Docker et Docker Compose installés et accessibles dans le `PATH`. Les sources de `fichiers_sources/` sont copiées dans l'image au moment du build. Le dossier local `output/` est monté dans le conteneur : les résultats y restent disponibles après l'arrêt du service.

Choix d'implémentation

Certaines modalités de réalisation ne sont pas précisées explicitement dans le cahier des charges. Les choix suivants ont donc été retenus afin de respecter au mieux sa structure, la succession de ses traitements et la forme de ses livrables.

Format ODS et utilisation de LibreOffice

Le cahier des charges demande la production de classeurs comportant différentes feuilles, sans imposer explicitement de format de fichier.

Il impose en revanche, à plusieurs endroits, des noms de feuilles très longs, dont certains dépassent 31 caractères. Or Microsoft Excel limite la longueur du nom d'une feuille à 31 caractères.

Le format **ODS**, utilisé notamment par LibreOffice Calc, a donc été retenu afin de pouvoir conserver les noms demandés par le cahier des charges sans les tronquer ni les remplacer par des alias.

Exécution dans un environnement Docker

La génération des classeurs repose sur LibreOffice et son interface Python UNO (PyUNO).

L'utilisation directe de PyUNO depuis l'environnement Python installé sur la machine hôte peut poser des problèmes de compatibilité selon le système d'exploitation et la manière dont Python et LibreOffice ont été installés. LibreOffice embarque notamment ses propres composants et bibliothèques Python/UNO, qui ne sont pas nécessairement accessibles ou compatibles avec l'interpréteur Python utilisé par le projet.

Afin d'éviter que le résultat du traitement dépende de la configuration locale de la machine, le choix a donc été fait d'utiliser Docker comme environnement d'exécution de référence.

L'image Docker fournit notamment :

- une version déterminée de Python ;
- LibreOffice ;
- PyUNO et les bibliothèques nécessaires à son fonctionnement ;
- les dépendances Python du projet ;
- un environnement identique d'une exécution à l'autre.

Ce choix permet de limiter les problèmes liés aux différences entre systèmes d'exploitation ou installations locales et améliore la reproductibilité du traitement.

L'utilisation de Docker n'est donc pas une exigence du cahier des charges, mais un choix technique destiné à garantir un environnement stable pour la génération automatisée des classeurs LibreOffice.

Mise en base des données brutes

Le cahier des charges ne demande pas la création d'une base de données.

Une étape intermédiaire de mise en base **SQLite** a néanmoins été ajoutée au traitement des données brutes.

Cette étape facilite :

- la normalisation et la centralisation des données parsées ;
- la préparation des données nécessaires aux premiers travaux ;
- les contrôles de cohérence ;
- les vérifications indépendantes par requêtes SQL ;
- la reproductibilité des résultats.

La base constitue donc un outil interne de traitement (pour la réalisation des fichiers .csv des travaux préliminaires uniquement) et de contrôle. Elle ne remplace pas la chaîne de feuilles demandée par le cahier des charges.

Exécution séquentielle des traitements

Les différentes étapes de préparation, de contrôle, d'agrégation et de comparaison sont exécutées **séquentiellement, dans l'ordre présenté par le cahier des charges**.

Cette organisation a été retenue afin de conserver une correspondance aussi directe que possible entre :

données brutes travaux préliminaires feuilles intermédiaires contrôles
agrégations comparaisons

Elle facilite également la compréhension de la filiation des données et permet de rapprocher chaque étape du programme de la section correspondante du cahier des charges.

Reproduction du fonctionnement d'un traitement manuel sous tableur

Lorsque le cahier des charges demande de créer une nouvelle feuille à partir d'une feuille précédente, le choix a été fait de **reproduire ce fonctionnement dans le programme comme si le traitement était réalisé manuellement dans un tableur**.

Ainsi, une feuille dérivée récupère ses données depuis la feuille qui la précède dans la chaîne de traitement, et non directement depuis la base SQLite.

De la même manière, lorsque le cahier des charges prévoit qu'un calcul soit effectué dans une feuille, celui-ci est autant que possible matérialisé dans la feuille correspondante plutôt que remplacé par un calcul effectué entièrement en amont dans le code.

Ce choix permet de conserver une filiation visible et vérifiable entre les feuilles :

feuille source copie transformation calcul agrégation comparaison

La base de données reste utilisée comme support de traitement et de contrôle, mais elle ne court-circuite pas les étapes explicitement décrites par le cahier des charges.

Copie des feuilles et conservation des champs

À plusieurs reprises, le cahier des charges demande de créer une nouvelle feuille à partir d'une feuille précédente, puis énumère les champs devant y figurer.

Dans certains cas, cette liste omet certains champs présents dans la feuille source alors même que l'instruction demande une copie de celle-ci.

Cette situation a été interprétée comme une ambiguïté ou une omission dans la description du cahier des charges. Afin d'éviter toute perte d'information, le choix a été fait de **conserver l'intégralité des champs de la feuille source**, puis d'ajouter les éventuels champs calculés demandés.

Cette approche privilégie la conservation de l'information, la traçabilité et la continuité entre les différentes étapes du traitement.

Classeurs dédiés aux feuilles de comparaison

Certaines feuilles demandées par le cahier des charges comparent des données provenant de deux classeurs distincts, sans préciser dans lequel de ces classeurs la feuille de comparaison doit être ajoutée.

Afin de ne pas rattacher arbitrairement une comparaison à l'une de ses deux sources, le choix a été fait de créer, dans ces situations, **un classeur indépendant contenant la feuille de comparaison**.

Cette organisation permet de distinguer clairement les données sources des résultats issus de leur rapprochement.

Limites du périmètre

- Le workflow traite les sources de caisse EJ/Z présentes pour MASSENA et MATURIN sur 2023, 2024 et 2025.
 - Les seuils de validation sont volontairement liés au jeu de données actuel. Ajouter ou retirer des sources nécessite de mettre à jour ces invariants dans `src/scripts/reconstruire_base_751.py` et les volumes attendus dans `src/scripts/db_vers_csv_751.py`.
 - Les déclarations CA3 ne sont pas incluses dans les sources actuelles ; leur saisie reste externe au programme.
 - Le programme ne traite pas les FEC et ne constitue pas à lui seul une attestation de conformité fiscale ou juridique.
-

Fonctionnement général

fichiers_sources/

EJ*.TXT

fichiers Z1 *.CSV

> output/database/db.sqlite

fichiers Z2 *.CSV

v

output/travaux_preliminaires/*.csv

v

output/libreoffice/*.ods

L'orchestrateur réalise les dix étapes suivantes.

1. Reconstruction et validation de SQLite

Le programme parcourt récursivement `fichiers_sources/`, calcule l'empreinte SHA-256 de chaque source EJ/Z, puis construit une base temporaire neuve contenant six tables :

- `tickets` et `lignes_ticket` pour les journaux EJ ;
- `z1_entetes` et `z1_lignes` pour les synthèses Z1 ;
- `z2_entetes` et `z2_lignes` pour les transactions Z2.

Les fichiers sont lus en Windows-1252 (`cp1252`). Les EJ retenus correspondent au motif `EJ*.TXT` . Les CSV Z1 et Z2 sont reconnus par les préfixes de fichiers définis dans `src/shared/constantes.py` .

Le périmètre actuel contient 63 fichiers EJ et 449 fichiers CSV. Ces 512 sources sont toutes protégées par empreinte, même si seuls les rapports Z1 et Z2 sont chargés dans SQLite.

Avant publication, la base temporaire est contrôlée avec les invariants du jeu de données actuel :

Contrôle	Valeur attendue
Blocs EJ	2 511
Tickets de vente exportables	1 875
Lignes de ticket	4 131
Retours de vente <code>_R_F</code>	35
Total TTC des retours	-19 821,00 €
Entêtes / lignes Z1	96 / 3 936
Entêtes / lignes Z2	96 / 4 800
Erreurs de clés étrangères	0

Les 1 875 ventes se répartissent en 1 153 tickets MASSENA et 722 tickets MATURIN. Si une validation échoue, la base temporaire est supprimée et la base active reste

inchangée. Si tout est valide, elle remplace atomiquement `output/database/db.sqlite` .

Les montants monétaires sont calculés avec `Decimal` et conservés comme texte décimal dans SQLite. Les retours `_R_F` sont normalisés avec des montants négatifs.

2. Génération des CSV intermédiaires

SQLite est exporté vers `output/travaux_preliminaires/` sous la forme de 16 fichiers séparés par `|` et encodés en UTF-8 avec BOM :

- deux CSV d'entêtes EJ et deux CSV de lignes EJ, un par boutique ;
- six CSV Z1, un par boutique et par exercice ;
- six CSV Z2, un par boutique et par exercice.

Les exports EJ ne contiennent que les blocs `REG` et `_R_F` dont le numéro de ticket est renseigné. Les autres blocs restent présents dans SQLite. Les dates sont normalisées au format `YYYY-MM-DD` , les identifiants restent du texte et les montants sont écrits avec deux décimales.

3. Génération des classeurs EJ

LibreOffice Calc, piloté en mode headless par PyUNO, crée quatre classeurs :

- `TTS_EJ_ENTETES_TICKETS_MASSENA.ods` ;
- `TTS_EJ_ENTETES_TICKETS_MATURIN.ods` ;
- `TTS_EJ_LIGNES_TICKETS_MASSENA.ods` ;
- `TTS_EJ_LIGNES_TICKETS_MATURIN.ods` .

Ils comprennent les données sources ainsi que les feuilles de tri, cohérence, séquentialité, doublons, occurrences et agrégations prévues par le traitement.

4. Génération des classeurs Z2

Six classeurs `TTS_Z2_TransactionsMois_TOUS_<année>_<boutique>.ods` sont créés pour MASSENA et MATURIN sur 2023, 2024 et 2025.

Ils contiennent les transactions, la période de clôture extraite du nom du fichier, les tableaux croisés par nature et par mode (`Z` , `ZZ1` , `ZZ2`) ainsi que les comparaisons entre modes disponibles.

5. Rapprochements Z2 / EJ

Les classeurs d'entêtes EJ sont enrichis avec les encaissements mensuels. Six classeurs autonomes comparent ensuite les règlements EJ aux montants Z2 :

- mode ZZ1 pour MASSENA ;
- mode Z pour MATURIN ;
- un classeur par boutique et par exercice.

Le rapprochement porte sur les cartes, chèques et espèces. Une source mensuelle absente reste vide : aucune valeur n'est inventée.

6. Génération des classeurs Z1

Six classeurs TTS_Z1_SyntheseMois_TOUS_<année>_<boutique>.ods sont créés. Ils regroupent les synthèses mensuelles, occurrences, agrégations de CA et totaux par mode Z.

7. Rapprochements Z1 / EJ

Les recettes mensuelles HT, TVA et TTC sont ajoutées aux classeurs d'entêtes EJ. Six classeurs autonomes comparent ces recettes aux synthèses Z1 :

- mode ZZ1 pour MASSENA ;
- mode Z pour MATURIN ;
- un classeur par boutique et par exercice.

Les écarts portent sur le CA TTC, le hors taxe et la TVA. Les périodes absentes d'une source sont signalées dans la sortie console et leurs écarts restent vides.

8. Consolidation des recettes

recettes_mensuelles_tous_boutique_232425.ods joint les recettes mensuelles de MASSENA et MATURIN, puis calcule les totaux toutes boutiques.

9. Comparaison avec les CA3

CompareCA_Gesco_CA3.ods reprend les recettes reconstituées et prépare leur comparaison avec les déclarations CA3. Les colonnes CA3 restent vides tant que ces données externes ne sont pas fournies ; les formules d'écart sont conditionnelles à leur présence.

10. Rapport d'exécution

`output/rapport-d-execution.txt` est construit à partir des mesures collectées pendant le traitement et d'une relecture des fichiers ODS publiés. Pour chaque feuille, il indique notamment :

- le fichier et l'onglet produits ;
 - les sources immédiates ;
 - les nombres d'enregistrements lus, sélectionnés et écrits ;
 - les totaux des champs monétaires et quantitatifs pertinents.
-

Résultats

Après une exécution complète, les résultats actifs sont organisés ainsi :

```
output/  
  database/  
    db.sqlite  
  travaux_preliminaires/  
    16 fichiers CSV  
  libreoffice/  
    4 classeurs EJ  
    6 classeurs Z1  
    6 classeurs Z2  
    6 comparaisons Z1 / EJ  
    6 comparaisons Z2 / EJ  
    recettes_mensuelles_tous_boutique_232425.ods  
    CompareCA_Gesco_CA3.ods  
  rapport-d-execution.txt
```

Au début d'une nouvelle exécution, si `output/` n'est pas vide, son contenu est copié dans `output/_sauvegarde_<horodatage>/`. Les fichiers actifs sont ensuite régénérés ou remplacés. Les sources placées dans `fichiers_sources/` ne sont jamais modifiées.

Architecture du projet

<code>src/</code>	
<code>traitement.py</code>	Orchestration des dix étapes
<code>classes/</code>	Parseurs et modèles EJ/Z
<code>scripts/</code>	SQLite, CSV, ODS et rapprochements

shared/	Constantes et fonctions partagées
documentation	Documentation du projet
tests/unit/	Tests unitaires et tests d'orchestration
tests/integration/	Tests d'intégration
fichiers_sources/	Sources EJ/Z et documents du dossier 751
output/	Base, intermédiaires et résultats générés
Dockerfile	Environnement Python/LibreOffice/PyUNO
docker-compose.yml	Montage du dossier de sortie
pyproject.toml	Dépendances et commande `traitement`
uv.lock	Versions verrouillées

Les scripts de `src/scripts/` peuvent être exécutés séparément pour le développement, mais ils ont des dépendances d'ordre entre leurs fichiers d'entrée. Pour une production cohérente, utiliser le point d'entrée `traitement`.

Documentation

Le répertoire `documentation/` contient les documents de référence du projet, notamment :

- `README.pdf` : explique le fonctionnement du projet et les choix d'implémentation ;
 - `RAPPORT_CONFORMITE_CDC_751.pdf` : rapport de conformité au cahier des charges 751, produit par le skill `audit-conformite-cdc-751`, qui vérifie la que le projet respecte les exigences du cahier des charges et que la chaîne sources-SQLite-traitements-ODS est conforme ;
 - `ANALYSE_JUSTIFICATION_CDC_751.pdf` : analyse des écarts et justification des choix d'implémentation pour fournir des éléments de réponse aux questions du cahier des charges 751.
-

Tests

Lancer la suite complète :

```
uv run python -m pytest -v
```

La suite couvre les parseurs EJ/Z, la reconstruction SQLite, les contrats CSV, la création et l'enrichissement des feuilles ODS, les rapprochements, la consolidation et le rapport

d'exécution.

Installation de Docker si pas encore installé

Docker Compose est inclus avec les versions récentes de Docker Desktop. La commande utilisée par le projet est donc :

```
docker compose
```

et non l'ancienne commande `docker-compose` .

Linux

Sur une distribution Debian/Ubuntu, Docker peut être installé avec :

```
sudo apt update  
sudo apt install -y docker.io docker-compose-v2
```

Démarrer et activer Docker :

```
sudo systemctl enable --now docker
```

Pour pouvoir utiliser Docker sans `sudo` :

```
sudo usermod -aG docker "$USER"
```

Il faut ensuite fermer puis rouvrir la session utilisateur.

Vérification :

```
docker --version  
docker compose version
```

macOS

Le moyen le plus simple est d'installer Docker Desktop avec Homebrew :

```
brew install --cask docker
```

Puis lancer Docker Desktop :

```
open -a Docker
```

Vérification :

```
docker --version  
docker compose version
```

Docker Desktop doit être lancé avant d'exécuter le traitement.

Windows

Depuis PowerShell, Docker Desktop peut être installé avec `winget` :

```
winget install -e --id Docker.DockerDesktop
```

Docker Desktop utilise généralement WSL 2 comme environnement d'exécution. Si WSL n'est pas encore installé :

```
wsl --install
```

Un redémarrage de Windows peut être nécessaire.

Docker Desktop peut ensuite être lancé depuis le menu Démarrer ou avec :

```
Start-Process "C:\Program Files\Docker\Docker\Docker Desktop.exe"
```

Vérification :

```
docker --version  
docker compose version
```

Docker Desktop doit être lancé avant d'exécuter le traitement.

Exécution locale

L'exécution hors Docker nécessite :

- Python 3.12 ou supérieur ;
- `uv` ;
- LibreOffice Calc avec PyUNO ;
- l'exécutable `soffice` accessible dans le `PATH` , ou une installation LibreOffice détectable.

Installation des dépendances Python verrouillées :

```
uv sync --frozen
```

Traitement complet avec les chemins par défaut :

```
uv run traitement
```

Les valeurs par défaut sont :

- sources : `fichiers_sources/` ;
- base : `output/database/db.sqlite` ;
- CSV intermédiaires : `output/travaux_preliminaires/` ;
- classeurs : `output/libreoffice/` .

Pour utiliser d'autres chemins :

```
uv run traitement \
chemin/vers/les_sources \
chemin/vers/db.sqlite \
--staging chemin/vers/les_csv \
--libreoffice chemin/vers/les_ods
```

`--travaux-preliminaires` est un alias de `--staging` . Le rapport d'exécution et la sauvegarde de début de traitement restent associés au dossier `output/` du projet.

Afficher l'aide :

```
uv run traitement --help
```